

Pandas & scikit-learn Patterns You'll See This Course — Session 4

Machine Learning in Geoscience — pre-session reference · read before Session 4

Session 3 trained one model per horizon on one fixed split. Session 4 asks a harder question — "which hyper-parameters, chosen honestly?" — which brings in cross-validation and search machinery. The new patterns are mostly about generating many train/val splits and many model configurations without writing them out by hand.

1. Time-aware cross-validation

`TimeSeriesSplit(n_splits=3, gap=H)`

Builds several train/validation folds automatically, each one using *only past* data to predict a later block — an "expanding window." Unlike ordinary k-fold, it never puts future rows in a training fold. `gap` leaves a buffer of `H` rows at each boundary, same purpose as the purge gap from Session 2.

`for i_tr, i_va in cv.split(X_tr):`

`cv.split(X)` is a generator: each iteration hands you a pair of integer-index arrays for that fold's train and validation rows. You then use them the same way as any other index array, e.g. `X_tr.iloc[i_tr]`.

```
cv = TimeSeriesSplit(n_splits=3, gap=6)
for i_tr, i_va in cv.split(X_tr):
    fold_X_train = X_tr.iloc[i_tr]
    fold_X_val   = X_tr.iloc[i_va]
```

2. Randomized hyper-parameter search

`param_distributions = {...}`

A plain dict naming each hyper-parameter and the list of candidate values to try — not a single configuration, but the whole space to search over.

`RandomizedSearchCV(model, param_distributions, cv=cv, scoring='neg_mean_absolute_error')`

Tries a fixed number of random combinations from that space, scoring each one with the folds from `cv`, and keeps the best. It's a model-like object itself: call `.fit(X_train, y_train)` on it.

Why scores are *negative* MAE

scikit-learn's search tools always maximize a score. Error metrics like MAE are meant to be minimized, so scikit-learn negates them internally (`neg_mean_absolute_error`) so "bigger is better" holds everywhere. A score of `-0.42` means an MAE of `0.42`.

`search.best_params_` / `search.best_estimator_`

After fitting, these hold the winning hyper-parameter dict and a ready-to-use model already trained with it.

Before Session 4 — a 10-minute self-check

- On a toy time series, create a `TimeSeriesSplit(n_splits=3)` and print the row ranges of each fold's train/validation split.
- Write a one-line `lambda` factory for any scikit-learn model and confirm calling it twice gives two distinct, unfitted objects.
- If something above is still unclear, that's fine — flag it at the start of the session instead of getting stuck mid-lecture.

3. Model factories

`factory = lambda: RandomForestRegressor(...)`

A zero-argument function that *builds a fresh, untrained model* each time it's called. Needed because a fitted model shouldn't be refit and reused — calling `factory()` twice gives two independent models, so the same configuration can be safely refit at every horizon or every fold.

```
factories = [
    ('Random Forest', lambda:
     RandomForestRegressor(n_estimators=200, random_state=SEED)),
    ('Boosting',      tuned_booster),    # an ordinary function
                                         works too
]
for name, factory in factories:
    model = factory()                    # brand-new, untrained
    model.fit(X_train, y_train)
```

4. Timing and memory bookkeeping

`t0 = time.time(); ...; elapsed = time.time() - t0`

The standard way to measure how long a step took — record the clock time before and after, subtract. Used here to report fit time as part of a model's operational cost.

`del model`

Explicitly drops the reference to a (possibly large) fitted model once its predictions are saved, so memory doesn't pile up across many horizons and many search candidates in the same run.

5. Falling back when a library is missing

`if HAS_LGBM: ... elif HAS_XGB: ... else: ...`

Boolean flags set once at import time (e.g. `HAS_LGBM = True` only if `import lightgbm` succeeded) let the same notebook run on any machine: use the best available boosting library, or fall back to scikit-learn's own `GradientBoostingRegressor` if neither is installed. You'll see this branch every time a model is constructed in this session.

Machine Learning in Geoscience — Session 4 pre-read. This sheet covers recognition only; the course notebook is where these patterns get used for real, with full explanations in context.